

Advanced Deep Knowledge Interview Questions for Senior Developers

This document contains advanced interview questions designed to evaluate in-depth technical understanding, engineering maturity, production debugging capability, distributed systems thinking, and real-world software architecture knowledge.

JVM Internals Deep Questions

1. Why is String stored in String Pool?
2. Why does new String() create a separate object?
3. Why is HashMap capacity usually a power of 2?
4. Why does ConcurrentHashMap disallow null keys and values?
5. Why does ArrayList resizing impact performance?
6. Why are fail-fast iterators considered best-effort only?

Garbage Collection & Memory

1. Why is Full GC dangerous in low-latency systems?
2. Why can memory usage increase even after GC?
3. Why are large object allocations expensive?
4. What causes memory fragmentation?
5. How do retained references create memory leaks?
6. How would you analyze a heap dump?

Spring / Enterprise Java

1. Why does self-invocation break @Transactional?
2. Why may @Transactional not rollback sometimes?
3. What thread-safety issues exist in singleton beans?
4. Difference between BeanFactory and ApplicationContext.
5. Why does lazy loading create N+1 query problems?
6. How does Spring AOP work internally?

REST APIs & Distributed Systems

1. Why should GET requests not modify state?
2. Why are retries dangerous in payment systems?
3. Why is exactly-once delivery difficult?
4. What is eventual consistency?
5. How do idempotency keys work?
6. What problems happen during network partition?

Kafka & Messaging Systems

1. Why is Kafka fast?
2. What causes consumer lag?
3. What happens if consumer crashes before offset commit?
4. Why are ordering guarantees difficult in distributed messaging?
5. Difference between at-most-once and at-least-once delivery.
6. How does partition rebalancing affect systems?

Database Internals

1. Why can UUID hurt database performance?
2. Why does OFFSET pagination become slow?
3. Why can OR conditions prevent index usage?
4. Why should transactions be short?
5. Difference between optimistic and pessimistic locking.
6. How does MVCC work?

Operating Systems & Linux

1. Difference between process and thread.
2. Why is context switching expensive?
3. What happens when CPU reaches 100% utilization?
4. Difference between IO-bound and CPU-bound workloads.
5. How does virtual memory work?

6. Why can excessive thread creation hurt performance?

Networking Questions

1. Why is TCP handshake necessary?
2. Why is HTTPS slower than HTTP?
3. What causes socket timeout?
4. Difference between TCP and UDP.
5. How does DNS resolution work?
6. What is connection pooling and why is it important?

Architecture & System Design

1. Why is caching difficult?
2. Why are distributed systems fundamentally unreliable?
3. Why can monoliths sometimes be better than microservices?
4. What tradeoffs exist in eventual consistency?
5. How would you design a scalable payment system?
6. What causes cascading failures in distributed systems?

BFSI / Banking Domain Deep Questions

1. Why can banking systems not blindly use async processing?
2. Why do reconciliation systems exist even with ACID transactions?
3. Why is duplicate payment prevention difficult?
4. Why do financial systems avoid floating point arithmetic?
5. How do settlement systems work?
6. Why are audit trails critical in BFSI systems?

Production Debugging & Real Incidents

1. Why does issue reproduce only in production?
2. What causes sudden API latency spikes?
3. Why can system fail at 100K users despite working at 1K users?

4. How do connection pool leaks happen?
5. How would you debug intermittent failures?
6. What was the toughest production bug you fixed?

Engineering Maturity Questions

1. Explain one design decision and its tradeoffs.
2. Describe a bug whose root cause surprised you.
3. What part of your current system worries you technically?
4. What engineering compromise did you disagree with?
5. How do you evaluate scalability risks?
6. What technical debt is most dangerous?

The strongest senior engineers are usually identified not by syntax knowledge, but by their understanding of failure modes, performance tradeoffs, debugging ability, scalability concerns, and production system behavior.